```
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, Flatten, Dropout

base_model = VGG16(include_top=False, input_shape=(224, 224, 3),
weights='imagenet')
for layer in base_model.layers:
    layer.trainable = False

x = base_model.output
x = Flatten()(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
output = Dense(1, activation='sigmoid')(x)

model = Model(inputs=base_model.input, outputs=output)
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
```

```
=======
```

Transfer Learning with Pretrained VGG16 (Cats vs Dogs)

Objective

Use a **pretrained VGG16 model** (trained on ImageNet) and perform **binary classification** (Cats vs Dogs) using **Transfer Learning**.

Question 5: Transfer Learning with Pretrained VGG16 (Cats vs Dogs)

Objective

Use a **pretrained VGG16 model** (trained on ImageNet) and perform **binary classification** (Cats vs Dogs) using **Transfer Learning**.

Code Explanation (Table Format)

Code	Purpose	Explanation
`from tensorflow.keras.applications import VGG16`	Import VGG16	Loads pretrained VGG16 model.
`from tensorflow.keras.models import Model`	Import Model Class	Used to create a custom model.
`from tensorflow.keras.layers import Dense, Flatten, Dropout`	Import Layers	Used to build custom classification layers.
`VGG16(...)`	Load Pretrained Model	Loads VGG16 trained on ImageNet.
`include_top=False`	Remove Original Classifier	Excludes VGG16's 1000-class output layer.
`input_shape=(224,224,3)`	Input Size	VGG16 expects RGB images of size 224×224.
`weights='imagenet'`	Pretrained Weights	Uses weights learned from ImageNet dataset.
`for layer in base_model.layers:`	Access Layers	Iterates through all VGG16 layers.
`layer.trainable=False`	Freeze Layers	Prevents pretrained weights from changing.
`base_model.output`	Feature Maps	Takes output from VGG16 feature extractor.
`Flatten()`	Flatten Features	Converts 3D feature maps into 1D vector.
`Dropout(0.5)`	Reduce Overfitting	Randomly disables 50% neurons during training.
`Dense(128, activation='relu')`	Hidden Layer	Learns task-specific features.

`Dense(1, activation='sigmoid')`	Output Layer	Produces binary output (Cat or Dog).
`Model(inputs, outputs)`	Create Final Model	Combines VGG16 and custom layers.
`optimizer='adam'`	Optimizer	Updates model weights.
`loss='binary_crossentropy'`	Loss Function	Used for binary classification.
`metrics=['accuracy']`	Evaluation Metric	Measures prediction accuracy.

What is Transfer Learning?

Term	Meaning
Transfer Learning	Reusing a pretrained model for a new task.
Pretrained Model	Model already trained on a large dataset.
Fine-Tuning	Adapting pretrained features to a new dataset.

Example

Original Training	New Task
ImageNet (1.4M Images, 1000 Classes)	Cats vs Dogs (2 Classes)

Instead of training from scratch, we reuse VGG16 knowledge.

VGG16 Architecture Flow

Input Image (224×224×3)
↓
Pretrained VGG16 Layers
↓
Feature Maps
↓

Flatten
↓
Dropout(0.5)
↓
Dense(128, ReLU)
↓
Dense(1, Sigmoid)
↓
Cat / Dog

Why `include_top=False`?

Value	Meaning
`True`	Keeps original 1000-class ImageNet classifier
`False`	Removes original classifier and allows custom classifier

For Cats vs Dogs, we need our own output layer.

Why Freeze Layers?

layer.trainable = False

Benefit	Explanation
Faster Training	Fewer parameters updated
Less Data Required	Uses already learned features
Better Accuracy	Retains ImageNet knowledge

Why Sigmoid?

Dense(1, activation='sigmoid')

Output	Meaning

0	Cat
1	Dog
0.85	85% confidence Dog

Sigmoid produces values between **0 and 1**.

Why Binary Crossentropy?

loss='binary_crossentropy'

Classification Type	Loss Function
Binary (2 Classes)	Binary Crossentropy
Multi-Class (>2)	Categorical Crossentropy

Key Parameters Summary

Parameter	Value	Purpose
Input Shape	224×224×3	VGG16 image size
Weights	ImageNet	Pretrained knowledge
Frozen Layers	All VGG16 Layers	Preserve learned features
Dropout	0.5	Reduce overfitting
Hidden Neurons	128	Learn task-specific patterns
Output Neurons	1	Binary classification
Activation	Sigmoid	Probability output
Optimizer	Adam	Weight optimization
Loss	Binary Crossentropy	Binary classification loss

Interview Questions

Question	Answer
What is VGG16?	A pretrained CNN model with 16 learnable layers trained on ImageNet.
What is Transfer Learning?	Reusing a pretrained model for a new task.
Why use `weights='imagenet'`?	To leverage features learned from millions of images.
Why use `include_top=False`?	To remove the original classifier and add a custom one.
Why freeze layers?	To preserve pretrained features and reduce training time.
Why use Sigmoid?	Suitable for binary classification.
Why use Binary Crossentropy?	It is the standard loss function for binary classification problems.

One-Line Summary

Question	Answer
What does this code do?	It uses Transfer Learning with a pretrained VGG16 model and adds custom layers to classify images as either Cat or Dog.

===============